

Technical Assignment

Web Security & JavaScript DOM Manipulation

Mata Kuliah: Web Application Development

Kelas: WAD05

Nama: MUH. AWALUDDIN

NIM: 25120300003

Implementasi: Pokémon Card Marketplace - Live Search

1. Perbedaan innerHTML dan textContent

innerHTML dan textContent sama-sama bisa digunakan untuk mengubah isi elemen HTML, tetapi cara browser membacanya berbeda.

innerHTML

innerHTML membaca nilai yang diberikan sebagai HTML. Jika ada tag HTML di dalam string, tag tersebut akan diproses oleh browser.

```
const productName = document.getElementById("productName");
productName.innerHTML = "<strong>Pikachu Card</strong>";
```

Pada contoh tersebut, tulisan Pikachu Card akan tampil tebal karena tag `<strong>` diproses sebagai HTML. innerHTML cocok ketika memang membutuhkan markup HTML, tetapi harus digunakan dengan hati-hati jika datanya berasal dari pengguna.

textContent

textContent hanya menampilkan nilai sebagai teks biasa. Tag HTML yang ada di dalam nilai tidak akan diproses.

```
const productName = document.getElementById("productName");
productName.textContent = "<strong>Pikachu Card</strong>";
```

Hasilnya adalah teks `<strong>Pikachu Card</strong>` apa adanya. Jika hanya ingin menampilkan teks, terutama dari pengguna, API, atau database, textContent lebih aman dan lebih sesuai.

Ringkasnya:

Properti	Cara membaca isi	Penggunaan
innerHTML	Diproses sebagai HTML	Saat membutuhkan markup HTML yang aman/terpercaya
textContent	Diproses sebagai teks biasa	Saat hanya ingin menampilkan teks

2. Contoh Kasus Cross-Site Scripting (XSS)

Misalnya sebuah marketplace kartu Pokémon memiliki fitur ulasan produk. Pengguna bisa menulis komentar pada kartu yang dijual, kemudian komentar tersebut disimpan ke database dan ditampilkan kembali di halaman produk.

Masalah bisa muncul jika komentar pengguna langsung dimasukkan ke halaman menggunakan innerHTML tanpa sanitasi atau penanganan yang aman.

```
reviewContainer.innerHTML = userReview;
```

Bagaimana celahnya terjadi?

Aplikasi menganggap isi ulasan sebagai HTML. Jika ada pengguna yang memasukkan konten HTML berbahaya, browser bisa ikut memprosesnya ketika ulasan tersebut ditampilkan.

Proses eksploitasi

1. Penyerang mengirim ulasan yang berisi konten berbahaya.
2. Website menyimpan ulasan tersebut ke database tanpa penanganan yang aman.
3. Pengguna lain membuka halaman produk yang sama.
4. Ulasan diambil dari database dan ditampilkan sebagai HTML.
5. Browser pengguna memproses isi tersebut.
6. Script berbahaya dapat berjalan di halaman korban.

Jika konten berbahaya disimpan di database lalu dijalankan kembali saat pengguna lain membuka halaman, kasus tersebut termasuk Stored XSS.

Dampak

XSS dapat digunakan untuk mengubah isi halaman, menampilkan form login palsu, membaca data yang dapat diakses JavaScript, atau menjalankan aksi melalui sesi pengguna yang sedang aktif.

Cookie yang memakai atribut HttpOnly biasanya tidak bisa dibaca langsung oleh JavaScript. Namun, XSS tetap berbahaya karena script masih dapat berinteraksi dengan halaman dan melakukan request menggunakan sesi pengguna dalam kondisi tertentu.

Pencegahan

- Gunakan `textContent` jika hanya perlu menampilkan teks.
- Jangan langsung memasukkan input pengguna sebagai HTML.
- Validasi input sesuai kebutuhan.
- Gunakan sanitasi jika aplikasi memang harus menerima HTML.
- Gunakan `createElement()` dan `textContent` saat membuat elemen secara dinamis.

3. Live Search Pokémon Card Marketplace

Bagian ketiga dibuat dalam satu file HTML. Data kartu disimpan dalam array JavaScript yang berisi nama Pokémon, tipe, rarity, dan harga contoh. Pencarian dapat dilakukan berdasarkan nama, tipe, atau rarity dan hasilnya berubah langsung saat pengguna mengetik.

Alur program

1. Data disimpan dalam array `pokemonCards`.
2. Event input membaca perubahan pada kolom pencarian.
3. `Array.filter()` mencari data yang sesuai dengan keyword.
4. `renderResults()` menampilkan ulang hasil pencarian.
5. Elemen hasil dibuat menggunakan `document.createElement()`.

6. Teks dimasukkan menggunakan `textContent`.
7. Hasil lama dibersihkan sebelum hasil baru ditampilkan.

Contoh bagian filtering

```
searchInput.addEventListener("input", (event) => {
  const keyword = event.target.value.toLowerCase().trim();

  const filteredCards = pokemonCards.filter((pokemon) => {
    return (
      pokemon.name.toLowerCase().includes(keyword) ||
      pokemon.type.toLowerCase().includes(keyword) ||
      pokemon.rarity.toLowerCase().includes(keyword)
    );
  });

  renderResults(filteredCards);
});
```

Harga yang digunakan hanya data contoh, bukan harga pasar kartu Pokémon sebenarnya.

Cara Menjalankan

1. Buka folder project.
2. Buka file `index.html` menggunakan browser.
3. Ketik nama Pokémon, tipe, atau rarity pada kolom pencarian.
4. Hasil pencarian akan berubah secara otomatis.